PYTEST & API TESTING

Comprehensive Study Notes

From Fundamentals to Advanced API Automation

Topics: Pytest Core · Fixtures & Markers · Parametrize · Requests Library · Auth · Schema Validation · Framework Design

MODULE 1: Introduction to Pytest

1.1 What is Pytest?

Pytest is a powerful, mature, and feature-rich testing framework for Python. It is the most widely used testing tool in the Python ecosystem, favored for its simplicity, scalability, and rich plugin architecture.

Aspect	Description
Full Name	pytest (stylized in lowercase)
Type	Third-party testing framework for Python
Language	Python 3.7+
License	MIT License
Install	pip install pytest
Run Command	pytest (from terminal)
Config File	pytest.ini / pyproject.toml / setup.cfg

Key Characteristics

- Simple syntax — uses plain assert statements, no special APIs needed
- Auto-discovery — finds test files and functions automatically
- Fixtures — powerful dependency injection mechanism for test setup/teardown
- Markers — tag tests for selective execution
- Plugins — 1000+ plugins available (pytest-html, pytest-cov, pytest-xdist, etc.)
- Parametrize — run the same test with multiple data sets
- Excellent reporting — clear, readable failure messages with assertion introspection

1.2 Pytest vs unittest Comparison

Python ships with a built-in testing module called unittest. Pytest is a third-party alternative that is more Pythonic and developer-friendly.

Feature	pytest	unittest	Winner
Syntax	Plain assert	self.assertEqual() etc.	pytest
Setup/Teardown	Fixtures (yield)	setUp/tearDown methods	pytest
Test Discovery	Automatic	Requires TestCase class	pytest
Parametrize	@pytest.mark.parametrize	No built-in support	pytest
Failure Messages	Detailed introspection	Basic messages	pytest

Plugins	1000+ plugins	Limited	pytest
Fixtures Scope	function/class/module/session	Only per-method	pytest
Learning Curve	Easy	Moderate (Java-like)	pytest
Compatibility	Can run unittest tests too	Cannot run pytest tests	pytest

1.3 Pytest Advantages & Features

Core Advantages

- No boilerplate: Write test functions directly, no class inheritance required
- Smart assertion rewriting: Pytest rewrites assert statements to show detailed failure info
- Fixture injection: Dependencies automatically injected via function arguments
- Test isolation: Each test gets a clean fixture instance (by default)
- Parallel testing: Run tests in parallel with pytest-xdist
- HTML/XML reports: Generate reports with pytest-html plugin
- Coverage: Integrated code coverage with pytest-cov

1.4 Pytest Architecture

Pytest follows a plugin-based architecture. Even the built-in features are implemented as internal plugins, making it highly extensible.

Component	Role
Collection Phase	pytest discovers all test files and test items
Setup Phase	Fixtures are set up before each test
Execution Phase	Test functions are called and results recorded
Teardown Phase	Fixtures are cleaned up after each test
Reporting Phase	Results are displayed / written to file
conftest.py	Shared plugin/fixture file recognized at all directory levels
pytest.ini	Global configuration — markers, options, paths
Hooks	Extension points for customizing collection, execution, reporting

1.5 Installing Pytest

```
# Install pytest
pip install pytest

# Verify installation
pytest --version

# Install common plugins
pip install pytest-html          # HTML reports
```

```
pip install pytest-cov           # Code coverage
pip install pytest-xdist         # Parallel execution
pip install requests             # For API testing
```

1.6 Project Structure & Naming Conventions

Recommended Project Structure

```
my_project/
├── src/
│   └── app.py
├── tests/
│   ├── conftest.py          # Shared fixtures
│   ├── test_unit.py         # Unit tests
│   ├── test_integration.py  # Integration tests
│   └── api/
│       ├── conftest.py      # API-specific fixtures
│       └── test_users.py    # API tests
├── pytest.ini               # Pytest configuration
└── requirements.txt
```

Naming Conventions

Item	Convention
Test Files	test_<name>.py or <name>_test.py
Test Functions	test_<description>()
Test Classes	Test<Name> (no underscore, no __init__)
Test Methods (in class)	test_<description>(self)
conftest.py	Exact name, placed in relevant directory
Fixtures	Descriptive noun, e.g., auth_token, base_url

1.7 Pytest Discovery Mechanism

Pytest automatically discovers tests using these rules:

1. Start from the current directory (or specified path)
2. Recurse into subdirectories (unless norecursedirs configured)
3. Collect files matching test_*.py or *_test.py
4. Inside those files, collect functions prefixed with test_
5. Collect classes prefixed with Test (without __init__)
6. Collect methods inside Test classes prefixed with test_

 Tip: You can customize discovery paths in pytest.ini using testpaths = tests/

1.8 Running Tests from CLI

```
# Run all tests
pytest
```

```
# Run specific file
pytest tests/test_users.py

# Run specific test function
pytest tests/test_users.py::test_create_user

# Run specific test class
pytest tests/test_users.py::TestUserAPI

# Run with verbose output
pytest -v

# Run with print output visible
pytest -s

# Run with short traceback
pytest --tb=short

# Run tests matching keyword
pytest -k "login or register"

# Run tests with marker
pytest -m smoke
```

1.9 Pytest Command-Line Options

Option	Description
-v / --verbose	Show each test name and PASSED/FAILED status
-s	Disable output capture — print() statements show in terminal
--tb=short	Short traceback on failure (less noise)
--tb=long	Full traceback with locals (default)
--tb=no	No traceback at all
-k 'expr'	Run tests matching expression (name-based filter)
-m 'marker'	Run tests with specified marker
-x	Stop after first failure
--maxfail=N	Stop after N failures
-q / --quiet	Minimal output
--co / --collect-only	Show what tests would be collected, without running
--html=report.html	Generate HTML report (requires pytest-html)
--cov=src/	Run coverage on src/ directory
-n auto	Run tests in parallel (requires pytest-xdist)

1.10 First Pytest Test Case

```
# tests/test_first.py
```

```
def add(a, b):
    return a + b

def test_add_positive_numbers():
    result = add(3, 5)
    assert result == 8

def test_add_negative_numbers():
    result = add(-1, -2)
    assert result == -3

def test_add_returns_integer():
    result = add(2, 3)
    assert isinstance(result, int)
```

```
# Run it:
$ pytest tests/test_first.py -v

# Expected output:
tests/test_first.py::test_add_positive_numbers  PASSED
tests/test_first.py::test_add_negative_numbers  PASSED
tests/test_first.py::test_add_returns_integer   PASSED
3 passed in 0.05s
```

MODULE 2: Assertions, Fixtures & Markers

2.1 Pytest Assertions Using assert

Pytest uses Python's built-in assert statement for all test assertions. Unlike unittest, you do NOT need special methods like assertEqual(), assertTrue(), etc.

```
# Basic assertions
assert result == expected          # Equality
assert result != unexpected        # Inequality
assert result > 0                  # Comparison
assert result is not None          # Identity
assert 'key' in response.json()    # Membership
assert isinstance(result, dict)    # Type check
assert len(items) == 5             # Length
```

2.2 Assertion Introspection

Pytest rewrites assert statements at collection time to provide detailed failure messages automatically — no extra code needed.

```
# When this fails:
assert response.status_code == 200

# Pytest output:
AssertionError: assert 404 == 200
 +  where 404 = <Response [404]>.status_code
```

2.3 Custom Assertion Messages

```
# Add context to assertion failures
assert response.status_code == 200, \
    f"Expected 200, got {response.status_code}. URL: {response.url}"

assert 'id' in response.json(), \
    f"'id' key missing. Response: {response.json()}"
```

2.4 Multiple Assertions in Single Test

```
def test_create_user_response():
    response = requests.post(BASE_URL + '/users', json=payload)
    data = response.json()

    assert response.status_code == 201
    assert data['name'] == 'John'
    assert data['job'] == 'Engineer'
    assert 'id' in data
    assert 'createdAt' in data
```

 Note: If the first assertion fails, subsequent ones won't run. Use pytest-check plugin for soft assertions that continue after failure.

2.5 Test Class Organization

```
# Group related tests in a class
class TestUserAPI:

    def test_get_user_success(self):
        response = requests.get(f'{BASE_URL}/users/1')
        assert response.status_code == 200

    def test_get_user_not_found(self):
        response = requests.get(f'{BASE_URL}/users/999')
        assert response.status_code == 404

    def test_get_users_list(self):
        response = requests.get(f'{BASE_URL}/users')
        assert len(response.json()['data']) > 0
```

2.6 What are Fixtures?

Fixtures are reusable setup/teardown blocks that pytest automatically injects into test functions via function arguments. They handle:

- Setting up preconditions (e.g., creating a DB connection, auth token)
- Providing test data
- Cleaning up after tests (teardown)
- Sharing state across multiple tests

2.7 @pytest.fixture Decorator

```
import pytest
import requests

BASE_URL = 'https://reqres.in/api'

@pytest.fixture
def base_url():
    return BASE_URL

@pytest.fixture
def sample_user():
    return {"name": "John", "job": "QA Engineer"}

# Using fixtures in a test (inject as parameters)
def test_create_user(base_url, sample_user):
    response = requests.post(f'{base_url}/users', json=sample_user)
    assert response.status_code == 201
```

2.8 Fixture Scope

Scope controls how often a fixture is created and destroyed. Choose scope based on how expensive the setup is.

Scope	Created	Destroyed	Use Case

function (default)	Before each test	After each test	Fresh state per test
class	Before first test in class	After last test in class	Shared within class
module	Before first test in module	After last test in module	Shared within file
session	Once per entire test run	After all tests complete	DB connection, auth token

```
@pytest.fixture(scope='session')
def auth_token():
    # Login once for all tests
    response = requests.post(BASE_URL + '/login', json={
        'email': 'eve.holt@reqres.in',
        'password': 'cityslicka'
    })
    return response.json()['token']
```

2.9 Fixture Setup & Teardown with yield

```
@pytest.fixture
def db_connection():
    # SETUP — runs before test
    conn = create_db_connection()
    conn.begin_transaction()

    yield conn              # Test receives 'conn'

    # TEARDOWN — runs after test (even if test fails)
    conn.rollback()
    conn.close()

# For API tests — example with session
@pytest.fixture(scope='module')
def api_session():
    session = requests.Session()
    session.headers.update({'Content-Type': 'application/json'})
    yield session
    session.close()
```

2.10 Fixture Dependencies

```
# Fixtures can depend on other fixtures

@pytest.fixture(scope='session')
def base_url():
    return 'https://reqres.in/api'

@pytest.fixture(scope='session')
def auth_token(base_url):   # depends on base_url
    resp = requests.post(f'{base_url}/login', json={...})
    return resp.json()['token']

@pytest.fixture
def headers(auth_token):    # depends on auth_token
    return {'Authorization': f'Bearer {auth_token}'}

def test_profile(base_url, headers):
```

```
    resp = requests.get(f'{base_url}/user/2', headers=headers)
    assert resp.status_code == 200
```

2.11 conftest.py for Fixture Sharing

conftest.py is a special pytest file that is automatically loaded. Fixtures defined here are available to all tests in the same directory and subdirectories — no import needed.

conftest.py Location	Scope of Fixtures
Root / (project root)	Available to ALL tests
tests/conftest.py	Available to all tests in tests/
tests/api/conftest.py	Available to tests in tests/api/ only

```
# tests/conftest.py
import pytest, requests

BASE_URL = 'https://reqres.in/api'

@pytest.fixture(scope='session')
def base_url():
    return BASE_URL

@pytest.fixture(scope='session')
def auth_token(base_url):
    resp = requests.post(f'{base_url}/login',
                         json={'email': 'eve.holt@reqres.in',
                               'password': 'cityslicka'})
    return resp.json()['token']

@pytest.fixture(scope='session')
def api_headers(auth_token):
    return {'Authorization': f'Bearer {auth_token}',
            'Content-Type': 'application/json'}
```

2.12 Pytest Markers (@pytest.mark)

Markers are labels you attach to tests to categorize, skip, or modify their behavior.

Built-in Markers

```
import pytest

# Skip unconditionally
@pytest.mark.skip(reason='Feature not implemented yet')
def test_future_feature():
    pass

# Skip conditionally
import sys
@pytest.mark.skipif(sys.platform == 'win32', reason='Not supported on Windows')
def test_linux_only():
    pass

# Expected failure (xfail)
@pytest.mark.xfail(reason='Known bug #123')
```

```
def test_known_bug():
    assert 1 == 2   # This will be reported as XFAIL (expected), not FAILED
```

Custom Markers

```
# Apply custom markers to tests
@pytest.mark.smoke
def test_health_check():
    assert True

@pytest.mark.regression
@pytest.mark.slow
def test_full_workflow():
    pass

# Run only smoke tests
# pytest -m smoke

# Run smoke OR regression
# pytest -m 'smoke or regression'

# Run regression but NOT slow
# pytest -m 'regression and not slow'
```

Registering Custom Markers in pytest.ini

```
# pytest.ini
[pytest]
markers =
    smoke: Quick sanity tests run on every build
    regression: Full regression test suite
    slow: Tests that take more than 5 seconds
    auth: Tests requiring authentication
    crud: CRUD operation tests
```

 Warning: Without registering markers in pytest.ini, pytest shows a PytestUnknownMarkWarning. Always register custom markers.

MODULE 3: Parametrize & Data-Driven Testing

3.1 @pytest.mark.parametrize

Parametrize allows you to run the same test function multiple times with different input/output combinations — the foundation of data-driven testing.

```
# Syntax
@pytest.mark.parametrize('param_name', [value1, value2, value3])
def test_something(param_name):
    ...
```

3.2 Single Parameter Parametrization

```
import pytest, requests

BASE_URL = 'https://reqres.in/api'

@pytest.mark.parametrize('user_id', [1, 2, 3, 4, 5])
def test_get_user_exists(user_id):
    response = requests.get(f'{BASE_URL}/users/{user_id}')
    assert response.status_code == 200
    assert response.json()['data']['id'] == user_id
```

3.3 Multiple Parameter Combinations

```
@pytest.mark.parametrize('name, job, expected_status', [
    ('Alice',   'Engineer',  201),
    ('Bob',     'Designer',  201),
    ('Charlie', 'Manager',   201),
])
def test_create_user(name, job, expected_status):
    payload = {'name': name, 'job': job}
    response = requests.post(f'{BASE_URL}/users', json=payload)
    assert response.status_code == expected_status
    assert response.json()['name'] == name
```

3.4 Parametrize with IDs for Readability

```
@pytest.mark.parametrize('status_code, user_id', [
    (200, 1),
    (200, 2),
    (404, 999),
], ids=['user_1_found', 'user_2_found', 'user_999_not_found'])
def test_user_status(status_code, user_id):
    response = requests.get(f'{BASE_URL}/users/{user_id}')
    assert response.status_code == status_code

# Output with ids:
# test_user_status[user_1_found] PASSED
# test_user_status[user_2_found] PASSED
# test_user_status[user_999_not_found] PASSED
```

3.5 Reading Test Data from External Files

```
# test_data/users.json
[
  {"name": "Alice", "job": "Engineer", "expected": 201},
  {"name": "Bob",   "job": "Manager",  "expected": 201}
]

# test_users.py
import json

def load_test_data(filename):
    with open(f'test_data/{filename}') as f:
        return json.load(f)

user_data = load_test_data('users.json')

@pytest.mark.parametrize('payload', user_data,
    ids=[d['name'] for d in user_data])
def test_create_users_from_file(payload):
    response = requests.post(BASE_URL + '/users', json=payload)
    assert response.status_code == payload['expected']
```

3.6 Combining Parametrize with Fixtures

```
@pytest.fixture(scope='session')
def base_url():
    return 'https://reqres.in/api'

@pytest.mark.parametrize('endpoint, expected_count', [
    ('/users?page=1', 6),
    ('/users?page=2', 6),
])
def test_pagination(base_url, endpoint, expected_count):
    response = requests.get(base_url + endpoint)
    assert response.status_code == 200
    assert len(response.json()['data']) == expected_count
```

MODULE 4: Requests Library & HTTP Methods

4.1 Installing & Importing Requests

```
# Install
pip install requests

# Import
import requests

# Verify
import requests
print(requests.__version__)    # e.g., 2.31.0
```

4.2 HTTP Methods with Requests

Method	requests function
GET	requests.get(url, params=..., headers=...)
POST	requests.post(url, json=..., data=..., headers=...)
PUT	requests.put(url, json=..., headers=...)
PATCH	requests.patch(url, json=..., headers=...)
DELETE	requests.delete(url, headers=...)

GET Request

```
response = requests.get('https://reqres.in/api/users/1')
print(response.status_code)    # 200
print(response.json())         # {'data': {'id':1, 'email':...}}
```

POST Request

```
payload = {'name': 'Alice', 'job': 'Engineer'}
response = requests.post('https://reqres.in/api/users', json=payload)
print(response.status_code)    # 201
print(response.json())         # {'name':'Alice', 'job':'Engineer', 'id':...,
'createdAt':...}
```

PUT Request (Full Update)

```
payload = {'name': 'Bob', 'job': 'Senior Engineer'}
response = requests.put('https://reqres.in/api/users/2', json=payload)
print(response.status_code)    # 200
```

PATCH Request (Partial Update)

```
payload = {'job': 'Tech Lead'}
response = requests.patch('https://reqres.in/api/users/2', json=payload)
print(response.status_code)    # 200
```

DELETE Request

```
response = requests.delete('https://reqres.in/api/users/2')
print(response.status_code)    # 204 (No Content)
```

4.3 Response Object Attributes

Attribute / Method	Description & Example
response.status_code	HTTP status code: 200, 201, 404, 500
response.json()	Parse JSON body → Python dict/list
response.text	Response body as string
response.content	Response body as bytes
response.headers	Response headers as dict-like object
response.url	Final URL (after redirects)
response.elapsed	Time taken as timedelta object
response.ok	True if status_code < 400
response.cookies	Cookies returned by server
response.encoding	Response encoding (e.g., 'utf-8')

4.4 Practice APIs

API	Description & Endpoints
reqres.in	Fake REST API for testing. GET /users, POST /users, POST /login, DELETE /users/{id}
restful-booker.herokuapp.com	Booking API with auth. POST /auth, GET/POST/PUT/DELETE /booking
jsonplaceholder.typicode.com	Fake REST for posts, comments, users — read-only
httpbin.org	HTTP request & response testing service — useful for debugging

MODULE 5: Request Parameters & Response Validation

5.1 Passing Headers

```
headers = {
    'Content-Type': 'application/json',
    'Authorization': 'Bearer my_token_here',
    'Accept': 'application/json'
}

response = requests.get(BASE_URL + '/users', headers=headers)
```

5.2 Query Parameters

```
# Option 1: Inline in URL
response = requests.get('https://reqres.in/api/users?page=2&per_page=6')

# Option 2: params dict (recommended — handles URL encoding)
params = {'page': 2, 'per_page': 6}
response = requests.get('https://reqres.in/api/users', params=params)
# Sends: GET https://reqres.in/api/users?page=2&per_page=6
```

5.3 Path Parameters

```
# Path parameters are embedded in the URL string
user_id = 3
booking_id = 101

response = requests.get(f'{BASE_URL}/users/{user_id}')
response = requests.get(f'{BASE_URL}/bookings/{booking_id}')

# Dynamic construction in parametrized tests
@pytest.mark.parametrize('user_id', [1, 2, 3])
def test_get_user(user_id):
    url = f'{BASE_URL}/users/{user_id}'
    response = requests.get(url)
    assert response.status_code == 200
```

5.4 Request Body — JSON vs Form Data

```
# JSON body (most common for REST APIs)
# Sets Content-Type: application/json automatically
response = requests.post(url, json={'name': 'Alice', 'job': 'Engineer'})

# Form data (HTML form submissions, some legacy APIs)
# Sets Content-Type: application/x-www-form-urlencoded
response = requests.post(url, data={'username': 'user', 'password': 'pass'})
```

5.5 Reading Payload from JSON Files

```
# payloads/create_user.json
# {"name": "Alice", "job": "Engineer"}

import json
```

```
def load_payload(filename):
    with open(f'payloads/{filename}') as f:
        return json.load(f)

def test_create_user_from_file():
    payload = load_payload('create_user.json')
    response = requests.post(BASE_URL + '/users', json=payload)
    assert response.status_code == 201
```

5.6 Timeout Handling

```
# Set timeout in seconds — raises requests.Timeout if exceeded
response = requests.get(url, timeout=5)          # 5s for both connect & read
response = requests.get(url, timeout=(3, 10))    # 3s connect, 10s read

# Handle timeout in tests
import pytest
with pytest.raises(requests.Timeout):
    requests.get(url, timeout=0.001)             # Intentionally tiny timeout
```

5.7 Session Management with requests.Session()

```
# Session persists headers, cookies, and TCP connections across requests
session = requests.Session()

# Set common headers once
session.headers.update({
    'Content-Type': 'application/json',
    'Accept': 'application/json'
})

# All requests share the session's headers/cookies
r1 = session.get(BASE_URL + '/users')
r2 = session.post(BASE_URL + '/users', json={'name': 'Bob'})
r3 = session.delete(BASE_URL + '/users/1')

session.close()    # Always close when done
```

5.8 Session as Pytest Fixture

```
@pytest.fixture(scope='module')
def api_session():
    session = requests.Session()
    session.headers.update({'Content-Type': 'application/json'})
    yield session
    session.close()

def test_create_user(api_session):
    response = api_session.post('/users', json={'name': 'Alice'})
    assert response.status_code == 201
```

5.9 Comprehensive Response Validation

```
def test_get_user_full_validation():
    response = requests.get(f'{BASE_URL}/users/2')
```

```
    # Status code
    assert response.status_code == 200

    # JSON parsing
    data = response.json()

    # Top-level keys
    assert 'data' in data
    assert 'support' in data

    # Nested JSON navigation
    user = data['data']
    assert user['id'] == 2
    assert user['email'] == 'janet.weaver@reqres.in'
    assert user['first_name'] == 'Janet'

    # Response headers
    assert 'application/json' in response.headers['Content-Type']

    # Response time validation
    assert response.elapsed.total_seconds() < 2.0
```

5.10 List & Array Validation

```
def test_users_list():
    response = requests.get(f'{BASE_URL}/users?page=1')
    data = response.json()

    # Array length
    assert len(data['data']) == 6

    # All items have required keys
    for user in data['data']:
        assert 'id' in user
        assert 'email' in user
        assert '@' in user['email']    # Email format check

    # Total count
    assert data['total'] == 12
    assert data['total_pages'] == 2
```

MODULE 6: Authentication & API Chaining

6.1 Basic Authentication

```
# HTTP Basic Auth (username:password base64 encoded)
from requests.auth import HTTPBasicAuth

response = requests.get(url, auth=HTTPBasicAuth('admin', 'password123'))
# Shorthand:
response = requests.get(url, auth=('admin', 'password123'))
```

6.2 Bearer Token Authentication

```
token = 'abc123xyz'

# Set Authorization header manually
headers = {'Authorization': f'Bearer {token}'}
response = requests.get(url + '/protected', headers=headers)

# Using session for persistent auth
session = requests.Session()
session.headers['Authorization'] = f'Bearer {token}'
response = session.get(url + '/protected')
```

6.3 Token Generation from Login API

```
# Step 1: Login to get token
login_payload = {
    'email': 'eve.holt@reqres.in',
    'password': 'cityslicka'
}
login_response = requests.post(BASE_URL + '/login', json=login_payload)
assert login_response.status_code == 200

# Step 2: Extract token
token = login_response.json()['token']

# Step 3: Use token in subsequent requests
headers = {'Authorization': f'Bearer {token}'}
protected_response = requests.get(BASE_URL + '/users', headers=headers)
```

6.4 Session-Scoped Auth Token Fixture

```
# conftest.py
@pytest.fixture(scope='session')
def auth_token(base_url):
    """Login once, reuse token for all tests."""
    payload = {'email': 'eve.holt@reqres.in', 'password': 'cityslicka'}
    response = requests.post(f'{base_url}/login', json=payload)
    assert response.status_code == 200, f'Login failed: {response.text}'
    return response.json()['token']

@pytest.fixture(scope='session')
def auth_headers(auth_token):
    return {
        'Authorization': f'Bearer {auth_token}',
```

```
        'Content-Type': 'application/json'
    }
```

6.5 API Chaining

API chaining is the pattern of using the response from one API call as input to the next. Common example: login → get token → use token → create resource → verify resource.

```
def test_full_booking_flow():
    # Step 1: Authenticate
    auth_resp = requests.post(BASE_URL + '/auth', json={
        'username': 'admin', 'password': 'password123'
    })
    assert auth_resp.status_code == 200
    token = auth_resp.json()['token']

    # Step 2: Create booking
    booking_payload = {'firstname': 'Alice', 'lastname': 'Smith',
                       'totalprice': 200, 'depositpaid': True,
                       'bookingdates': {'checkin': '2024-01-01',
                                        'checkout': '2024-01-05'}}
    create_resp = requests.post(BASE_URL + '/booking', json=booking_payload)
    assert create_resp.status_code == 200
    booking_id = create_resp.json()['bookingid']

    # Step 3: Verify booking was created
    get_resp = requests.get(f'{BASE_URL}/booking/{booking_id}')
    assert get_resp.status_code == 200
    assert get_resp.json()['firstname'] == 'Alice'

    # Step 4: Delete booking (cleanup)
    headers = {'Cookie': f'token={token}'}
    del_resp = requests.delete(f'{BASE_URL}/booking/{booking_id}', headers=headers)
    assert del_resp.status_code == 201
```

MODULE 7: Schema Validation & Negative Testing

7.1 Introduction to JSON Schema

JSON Schema is a vocabulary for describing the structure of JSON data. It allows you to validate that API responses follow the expected structure and data types — not just specific values.

Schema Keyword	Purpose
type	Data type: string, number, integer, boolean, array, object, null
properties	Define expected keys in an object and their schemas
required	List of keys that MUST be present
items	Schema for array elements
minimum / maximum	Numeric range constraints
minLength / maxLength	String length constraints
pattern	Regex pattern the string must match
enum	List of allowed values
additionalProperties	Whether extra keys are allowed

7.2 Creating a JSON Schema

```
# Schema for a single user object
user_schema = {
    'type': 'object',
    'properties': {
        'id':         {'type': 'integer'},
        'email':      {'type': 'string', 'format': 'email'},
        'first_name': {'type': 'string'},
        'last_name':  {'type': 'string'},
        'avatar':     {'type': 'string'}
    },
    'required': ['id', 'email', 'first_name', 'last_name']
}
```

7.3 Installing & Using jsonschema

```
pip install jsonschema

from jsonschema import validate, ValidationError

def test_user_schema_valid():
    response = requests.get(f'{BASE_URL}/users/2')
    user_data = response.json()['data']

    # validate() raises ValidationError if schema doesn't match
    try:
        validate(instance=user_data, schema=user_schema)
    except ValidationError as e:
```

```
        pytest.fail(f'Schema validation failed: {e.message}')
```

7.4 Reusable Schema Validation Fixture

```
# conftest.py
from jsonschema import validate, ValidationError
import pytest

@pytest.fixture
def validate_schema():
    def _validate(data, schema):
        try:
            validate(instance=data, schema=schema)
        except ValidationError as e:
            pytest.fail(f'Schema validation failed:\n{e.message}')
    return _validate

# In test file:
def test_user_schema(validate_schema):
    response = requests.get(f'{BASE_URL}/users/1')
    validate_schema(response.json()['data'], user_schema)
```

7.5 Negative Testing Scenarios

Negative Test Type	What to Verify
Invalid payload	400 status, error message in response
Missing required fields	422/400 status, validation error
Wrong data types	400 status, type error message
Missing/invalid auth	401 Unauthorized
Invalid endpoint	404 Not Found
Server error simulation	500 status handling
Boundary values	Edge of valid range: min-1, max+1
Empty payload	400 status with meaningful error

```
# Negative test examples

def test_login_invalid_credentials():
    payload = {'email': 'invalid@test.com', 'password': 'wrong'}
    response = requests.post(BASE_URL + '/login', json=payload)
    assert response.status_code == 400
    assert 'error' in response.json()

def test_get_nonexistent_user():
    response = requests.get(f'{BASE_URL}/users/9999')
    assert response.status_code == 404

def test_unauthorized_access():
    response = requests.get(BASE_URL + '/protected-resource')
    assert response.status_code == 401
```

```
def test_empty_payload():
    response = requests.post(BASE_URL + '/login', json={})
    assert response.status_code == 400

def test_missing_required_field():
    payload = {'email': 'test@test.com'}  # password missing
    response = requests.post(BASE_URL + '/login', json=payload)
    assert response.status_code == 400
```

7.6 pytest.raises for Exception Testing

```
import pytest, requests

def test_connection_timeout():
    with pytest.raises(requests.exceptions.Timeout):
        requests.get('http://httpbin.org/delay/10', timeout=0.001)

def test_connection_error():
    with pytest.raises(requests.exceptions.ConnectionError):
        requests.get('http://nonexistent-domain-12345.com')
```

MODULE 8: conftest.py & Framework Architecture

8.1 Purpose of conftest.py

- Central place for fixtures shared across multiple test files
- No imports needed — pytest auto-discovers conftest.py at each directory level
- Can contain hooks (pytest_configure, pytest_runtest_setup, etc.)
- Can register plugins and custom markers
- Multiple conftest.py files at different levels — each applies to its directory

8.2 Production-Grade conftest.py

```
# tests/conftest.py
import pytest, requests, os

# ── Environment configuration ──────────────────────────
ENVIRONMENTS = {
    'dev':  'https://dev.api.example.com',
    'qa':   'https://qa.api.example.com',
    'prod': 'https://api.example.com',
}

@pytest.fixture(scope='session')
def env():
    return os.getenv('TEST_ENV', 'qa')

@pytest.fixture(scope='session')
def base_url(env):
    return ENVIRONMENTS[env]

# ── Auth ───────────────────────────────────────────────
@pytest.fixture(scope='session')
def auth_token(base_url):
    resp = requests.post(f'{base_url}/auth/login', json={
        'username': os.getenv('API_USER', 'admin'),
        'password': os.getenv('API_PASS', 'password123')
    })
    assert resp.status_code == 200, f'Login failed: {resp.text}'
    return resp.json()['token']

# ── Session ────────────────────────────────────────────
@pytest.fixture(scope='session')
def api_client(base_url, auth_token):
    session = requests.Session()
    session.base_url = base_url
    session.headers.update({
        'Authorization': f'Bearer {auth_token}',
        'Content-Type': 'application/json',
        'Accept': 'application/json'
    })
    yield session
    session.close()
```

8.3 Fixture Factories for Dynamic Data

```
@pytest.fixture
```

```
def make_user():
    """Factory fixture — returns a function to create users with custom data."""
    created_ids = []

    def _make_user(name='Default', job='Tester'):
        resp = requests.post(BASE_URL + '/users', json={'name': name, 'job': job})
        user_id = resp.json().get('id')
        created_ids.append(user_id)
        return resp.json()

    yield _make_user

    # Teardown: clean up all created users
    for uid in created_ids:
        requests.delete(f'{BASE_URL}/users/{uid}')

# Usage:
def test_create_multiple(make_user):
    alice = make_user('Alice', 'Developer')
    bob   = make_user('Bob',   'Manager')
    assert alice['name'] == 'Alice'
    assert bob['name']   == 'Bob'
```

MODULE 9: Logging & Utility Framework

9.1 Python Logging for API Tests

```
import logging

# Configure logger
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(name)s | %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S'
)

logger = logging.getLogger(__name__)

def test_get_user():
    logger.info('Sending GET /users/1')
    response = requests.get(BASE_URL + '/users/1')
    logger.info(f'Status: {response.status_code}')
    logger.debug(f'Response body: {response.json()}')
    assert response.status_code == 200
```

9.2 Log Levels

Level	When to Use
DEBUG (10)	Detailed debug info — request/response bodies, headers
INFO (20)	General flow — 'Sending request to /users', 'Login successful'
WARNING (30)	Unexpected but non-fatal — slow response time, retry
ERROR (40)	Test failures, unexpected status codes
CRITICAL (50)	Setup failures — cannot connect to API, auth failed

9.3 Request/Response Logging Utility

```
# utils/logger.py
import logging, requests

logger = logging.getLogger('api')

def log_request_response(response: requests.Response):
    """Log full request and response details."""
    req = response.request
    logger.info(f'REQUEST  → {req.method} {req.url}')
    if req.body:
        logger.debug(f'  Body: {req.body}')
    logger.info(f'RESPONSE ← {response.status_code}
({response.elapsed.total_seconds():.3f}s)')
    logger.debug(f'  Body: {response.text[:500]}')

# Use as fixture:
@pytest.fixture(autouse=True)
def log_api_calls(api_client, monkeypatch):
    original_request = api_client.request
```

```
    def logged_request(method, url, **kwargs):
        resp = original_request(method, url, **kwargs)
        log_request_response(resp)
        return resp
    monkeypatch.setattr(api_client, 'request', logged_request)
```

9.4 Reusable Assertion Helpers

```
# utils/assertions.py
import pytest
from jsonschema import validate, ValidationError

def assert_status(response, expected_status):
    assert response.status_code == expected_status, \
        f'Expected {expected_status}, got {response.status_code}.\nBody:
{response.text}'

def assert_json_key(response, key):
    data = response.json()
    assert key in data, f"Key '{key}' not found. Keys: {list(data.keys())}"

def assert_schema(data, schema):
    try:
        validate(instance=data, schema=schema)
    except ValidationError as e:
        pytest.fail(f'Schema validation error: {e.message}')

def assert_response_time(response, max_seconds=2.0):
    elapsed = response.elapsed.total_seconds()
    assert elapsed < max_seconds, \
        f'Response too slow: {elapsed:.2f}s (max: {max_seconds}s)'
```

9.5 Config File Management

```
# config/config.yaml
environments:
  qa:
    base_url: https://qa.api.example.com
    timeout: 10
  prod:
    base_url: https://api.example.com
    timeout: 5
credentials:
  username: admin
  password: secret123

# utils/config.py
import yaml, os

def load_config(env='qa'):
    with open('config/config.yaml') as f:
        config = yaml.safe_load(f)
    return config['environments'][env]

# In conftest.py
@pytest.fixture(scope='session')
def config():
    env = os.getenv('TEST_ENV', 'qa')
    return load_config(env)

@pytest.fixture(scope='session')
```

```
def base_url(config):
    return config['base_url']
```

9.6 Complete Framework Structure

```
api_test_framework/
├── config/
│   ├── config.yaml          # Environment URLs, credentials
│   └── schemas/             # JSON Schema files
│       ├── user_schema.json
│       └── booking_schema.json
├── test_data/
│   ├── create_user.json
│   └── booking_payloads.json
├── tests/
│   ├── conftest.py          # Session fixtures: base_url, auth, client
│   ├── test_users.py
│   ├── test_auth.py
│   └── test_bookings.py
├── utils/
│   ├── assertions.py        # Custom assertion helpers
│   ├── logger.py            # Logging utilities
│   └── config.py            # Config loader
├── pytest.ini               # Markers, options
├── requirements.txt
└── README.md
```

9.7 pytest.ini — Full Configuration

```
[pytest]
testpaths = tests
addopts = -v --tb=short
log_cli = true
log_cli_level = INFO
log_format = %(asctime)s | %(levelname)s | %(message)s
log_date_format = %H:%M:%S

markers =
    smoke: Quick sanity tests
    regression: Full regression suite
    auth: Authentication tests
    crud: CRUD operation tests
    slow: Tests >5 seconds
    negative: Negative/error scenario tests
```

 Summary: A well-structured API test framework combines: conftest.py for fixtures, utils/ for helpers, config/ for data, and pytest.ini for settings — giving you a maintainable, scalable, environment-aware automation suite.